

# Архитектура высокой доступности (HA) и восстановления (DR)

AI Архитектор



● REC

**Напишите «+» в чат, если меня слышно и видно**





AI Архитектор

## Лебедев Михаил

Руководитель направления, PhD

- с 2002 по 2012 системный администратор;
- с 2012 по 2018 системный архитектор, руководитель проектов, технический писатель, разработчик;
- с 2018 по 2026 Data Scientist;
- с 2022 Преподаватель (Доцент ВШЭ, OTUS)

 @MichaelSwan

# Правила вебинара



Активно участвуем



Задаем вопросы в чат



Вопросы вижу в чате,  
могу ответить не сразу



Запись вебинара придёт на почту

# Маршрут вебинара

Проектирование отказоустойчивых систем

Паттерны отказоустойчивости

Расчёт RTO/RPO

Примеры DRP планов

Рефлексия

# Инфраструктура

# Физические серверы

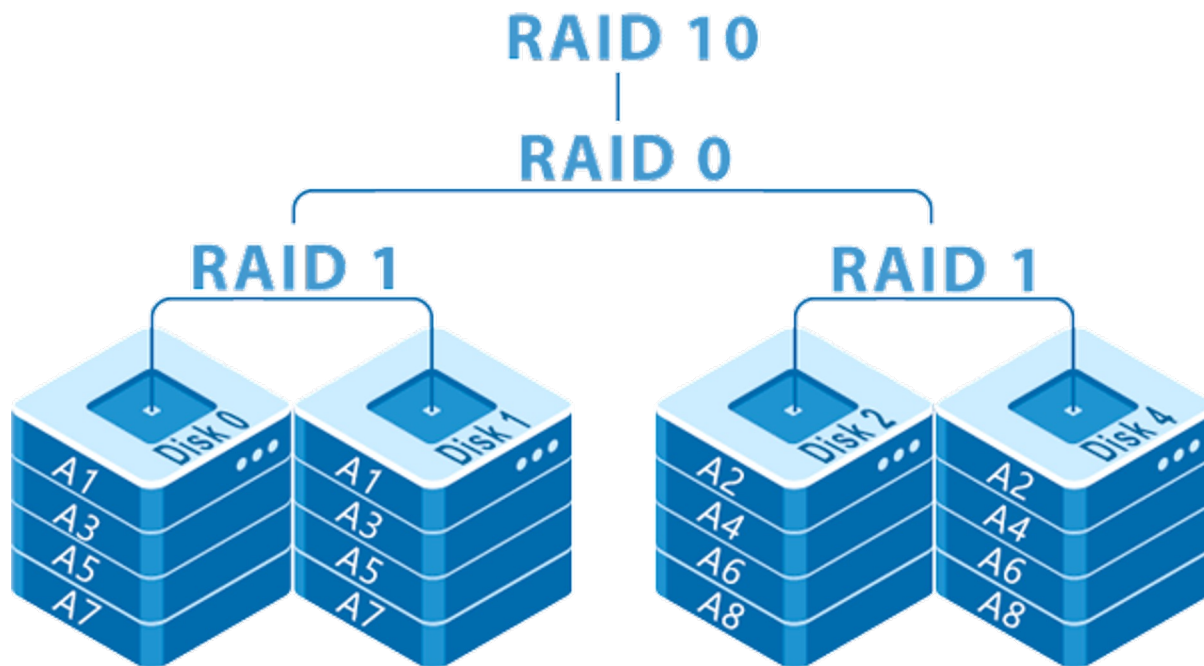


# Физические серверы

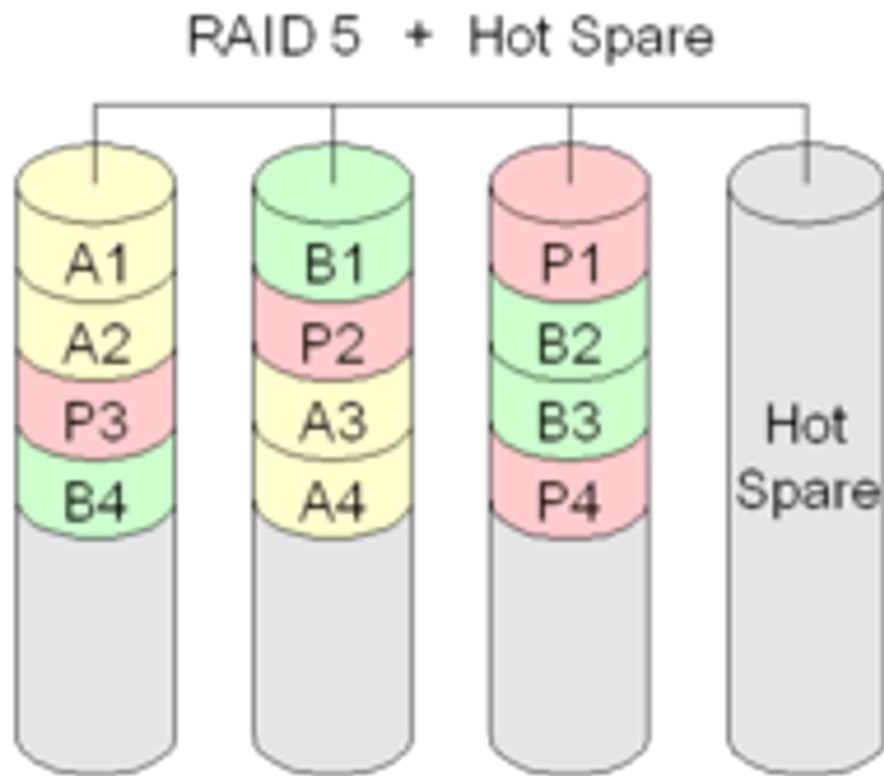




# Логические диски



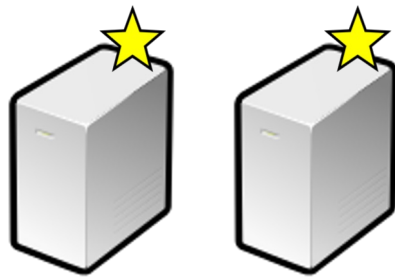
# Логические диски



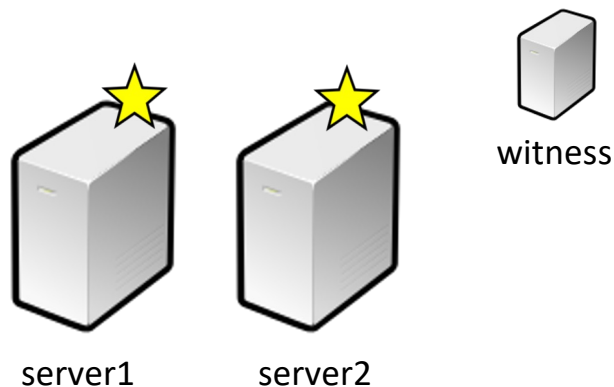
# Построение высокой доступности



# Построение высокой доступности

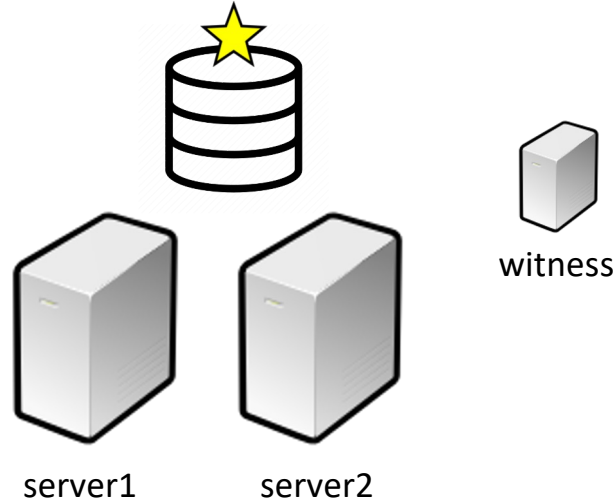


# Построение высокой доступности



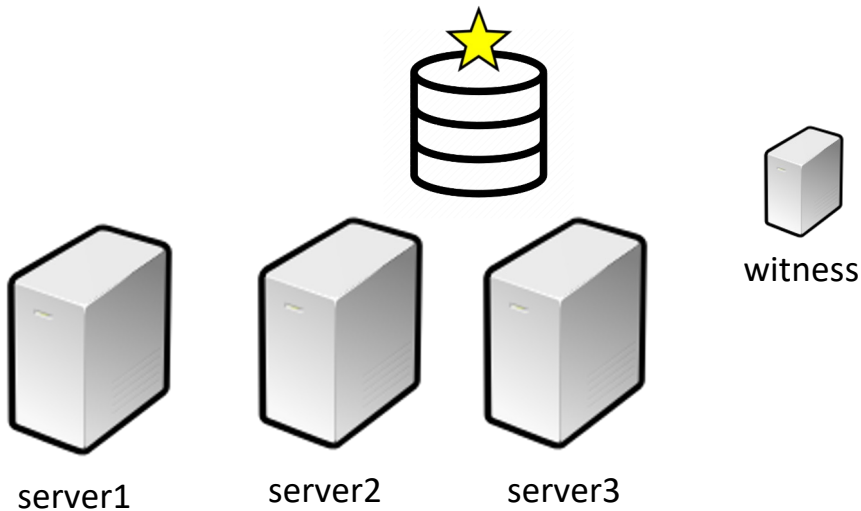
Высокая доступность (high availability) – быстрый перезапуск сервиса на другом сервере при сбое

# Построение высокой доступности



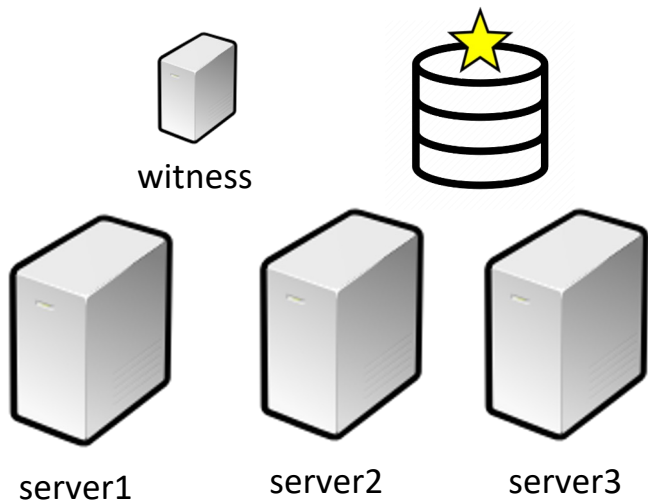
Отказоустойчивость (Fault Tolerance) - мгновенное переключение на дублирующую, синхронизированную копию без прерывания работы

# Построение высокой доступности

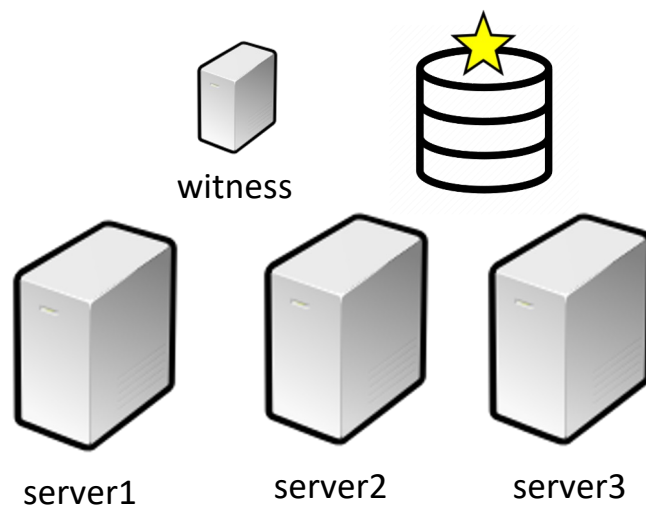


# Построение высокой доступности

ЦОД 1



ЦОД 2

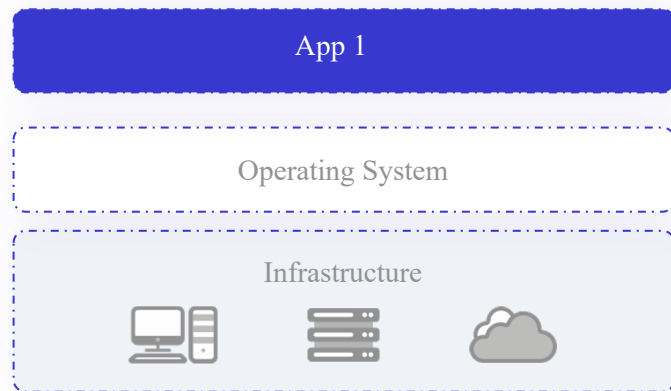




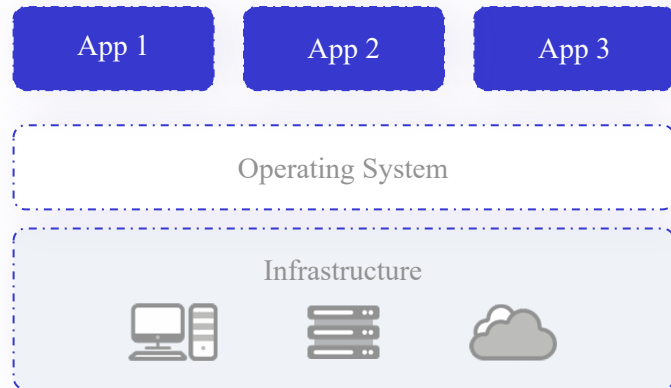
# Виртуализация

**RTO (Recovery Time Objective)** — Целевое время восстановления. Максимальное время, в течение которого система может быть недоступна без критических последствий для бизнеса.

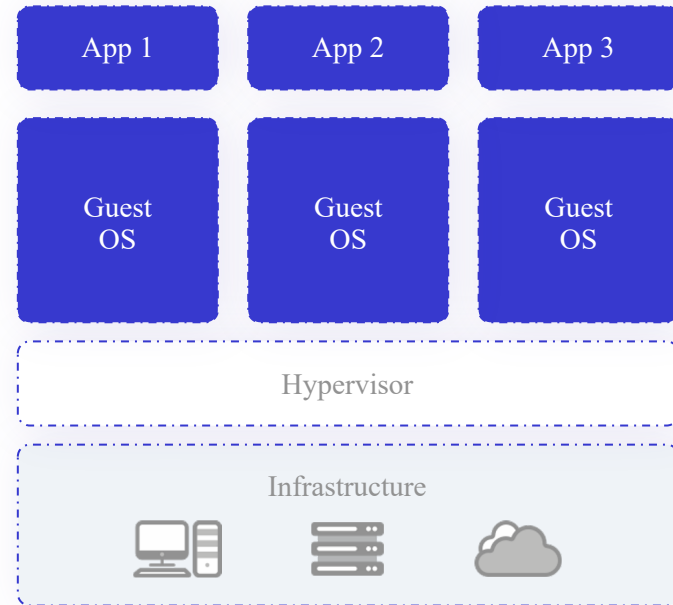
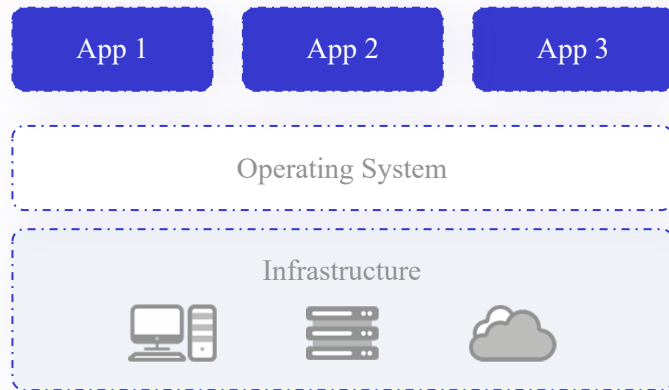
**RPO (Recovery Point Objective)** — Целевая точка восстановления. Максимальный промежуток времени, за который допускается потеря данных при сбое.



CPU  
RAM  
HDD  
RPO  
RTO  
COST



# Виртуализация



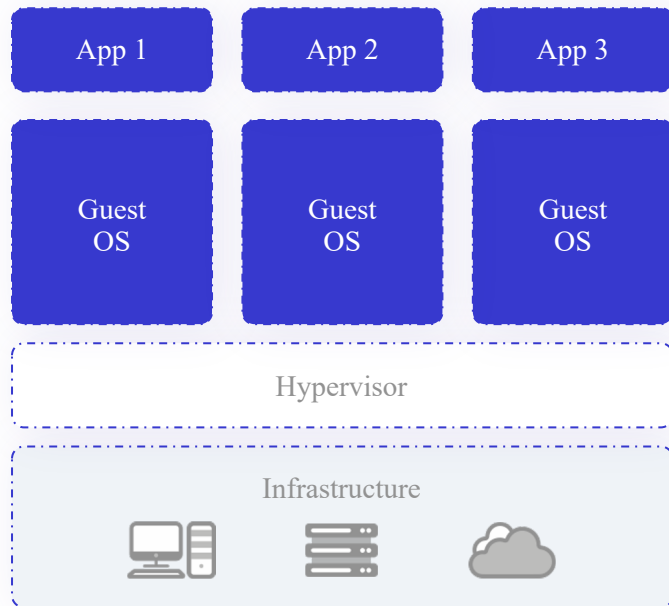
Machine Virtualization

CPU  
RAM  
HDD  
RPO  
RTO  
COST  
LIC

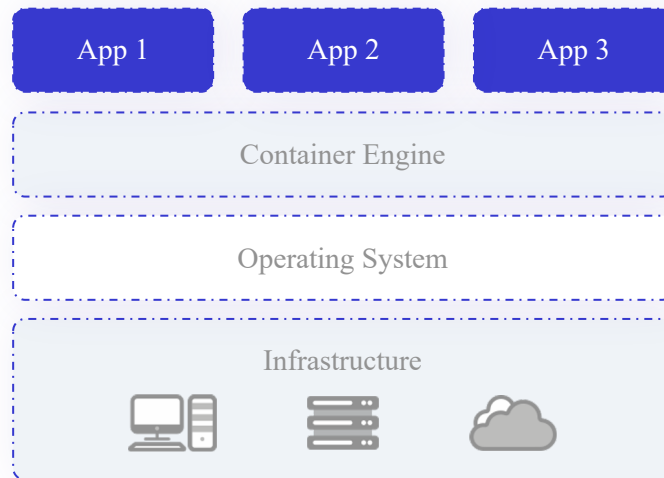
Гипервизоры: VMware ESXi, Microsoft Hyper-V, Citrix XenServer, KVM.



# Контейнеризация



Machine Virtualization



Docker Containerization

CPU  
RAM  
HDD  
RPOR  
TO  
COST  
LIC

# Ваши вопросы



если есть вопросы



если вопросов нет



**Высокая доступность (HA)**  
**Восстановление после сбоев (DR)**

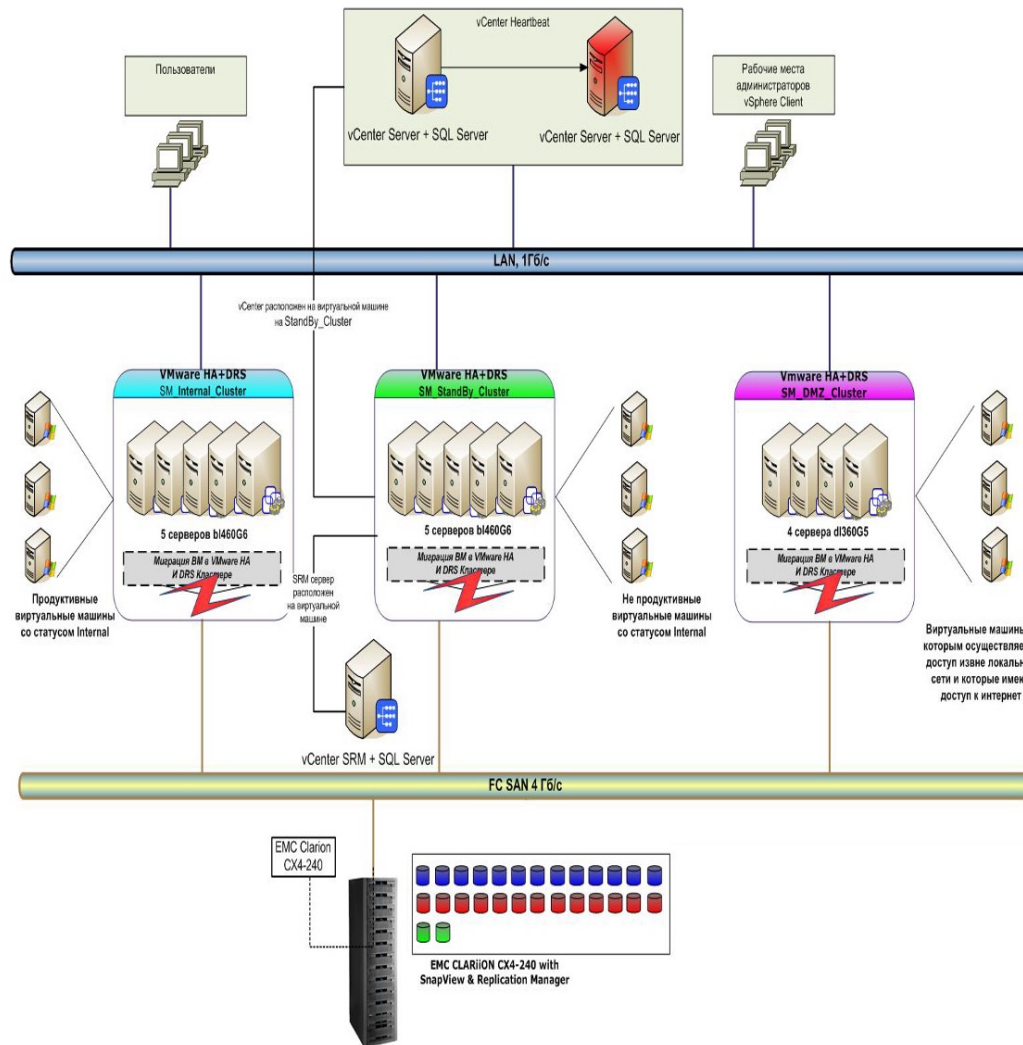
# HA vs DR

| Характеристика | Высокая доступность (HA)                                                                                    | Восстановление после сбоев (DR)                                                                                                                                                                                       |
|----------------|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Цель           | Предотвращение или минимизация простоев и потерь данных                                                     | Процесс восстановления работы системы после крупного сбоя (например, катастрофы, сбои на уровне дата-центра)                                                                                                          |
| Уровень        | Внутри одного дата-центра                                                                                   | Междатацентовой                                                                                                                                                                                                       |
| Восстановление | Мгновенное                                                                                                  | Время зависит от RTO                                                                                                                                                                                                  |
| Применяется    | Для критических сервисов                                                                                    | Для катастрофических сбоев                                                                                                                                                                                            |
| Способ         | Балансировка нагрузки<br>- Активный/резервный (Active/Passive) или активный/активный (Active/Active) режимы | - Активный/резервный (Active/Passive) — резервная копия, которая включается только при сбое<br>- Активный/активный (Active/Active) — оба дата-центра работают одновременно<br>- Репликация данных между дата-центрами |

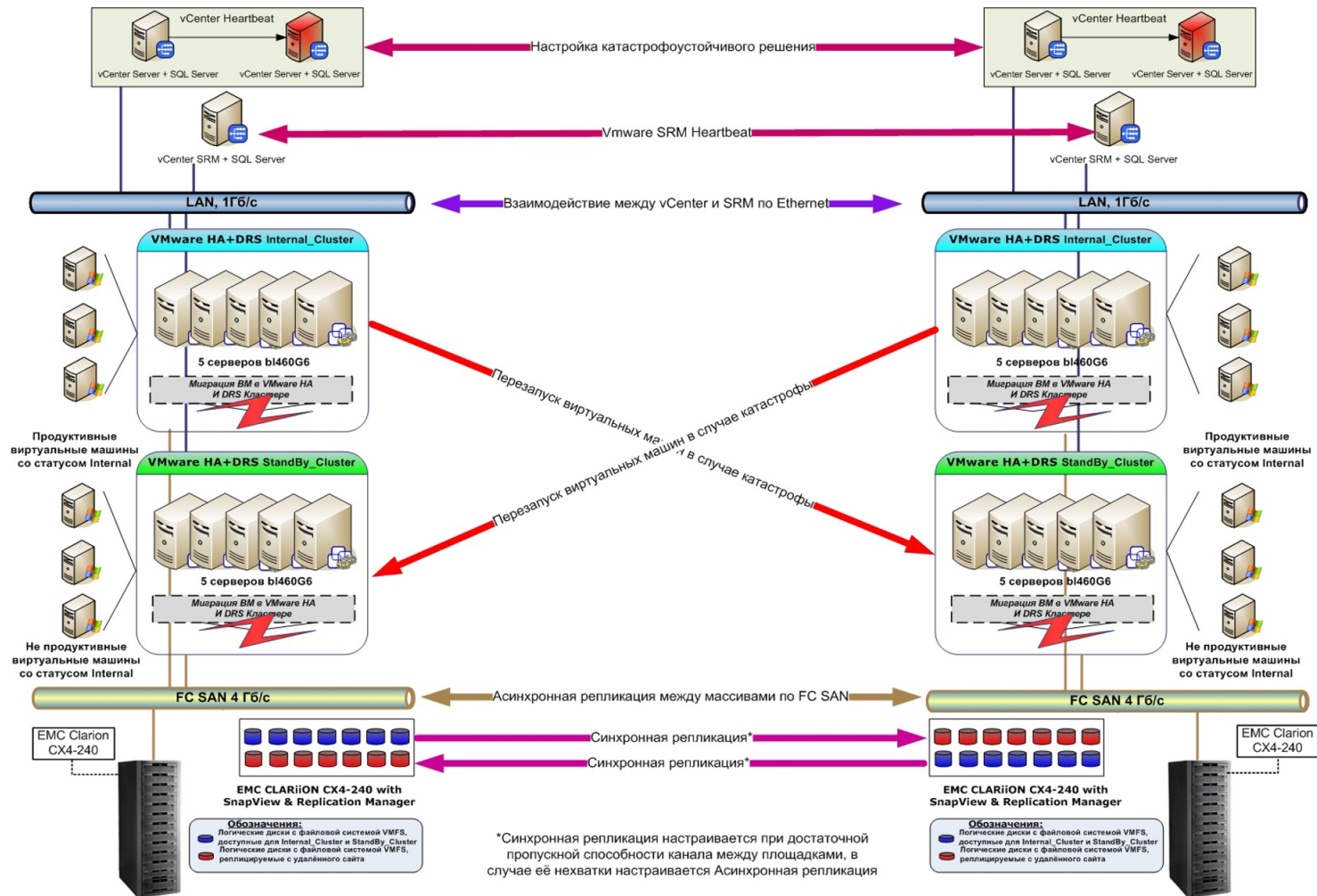
# HA vs DR

| Характеристика | Высокая доступность (HA)                                                                                                                                                                                                                                                                                            | Восстановление после сбоев (DR)                                                                                                                                                                                                                                                                                                         |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Способ         | <ul style="list-style-type: none"><li>- Редунданные (резервные) компоненты (серверы, сети, хранилища)</li><li>- Балансировщики нагрузки</li><li>- Системы мониторинга и автоматического переключения (failover)</li><li>- Распределённые базы данных</li><li>- Сервисы кэширования и сессионного хранения</li></ul> | <ul style="list-style-type: none"><li>- Балансировка трафика по регионам</li><li>- Синхронизация данных через репликацию</li></ul>                                                                                                                                                                                                      |
| Инструменты    | <ul style="list-style-type: none"><li>- Кластеризация (например, Kubernetes, Pacemaker)</li><li>- Репликация данных (MySQL, PostgreSQL, Redis)</li><li>- Многозональная/многорегиональная архитектура (AWS, Azure, GCP)</li><li>- Сервисы автоматического масштабирования</li></ul>                                 | <ul style="list-style-type: none"><li>- Репликация баз данных (MySQL, Oracle, MongoDB)</li><li>- Объектное хранилище с синхронизацией (S3, Blob Storage)</li><li>- DR-планы и тестирование (DR Testing)</li><li>- Системы автоматического переключения (Failover)</li><li>- Облачные DR-решения (AWS DR, Azure Site Recovery)</li></ul> |

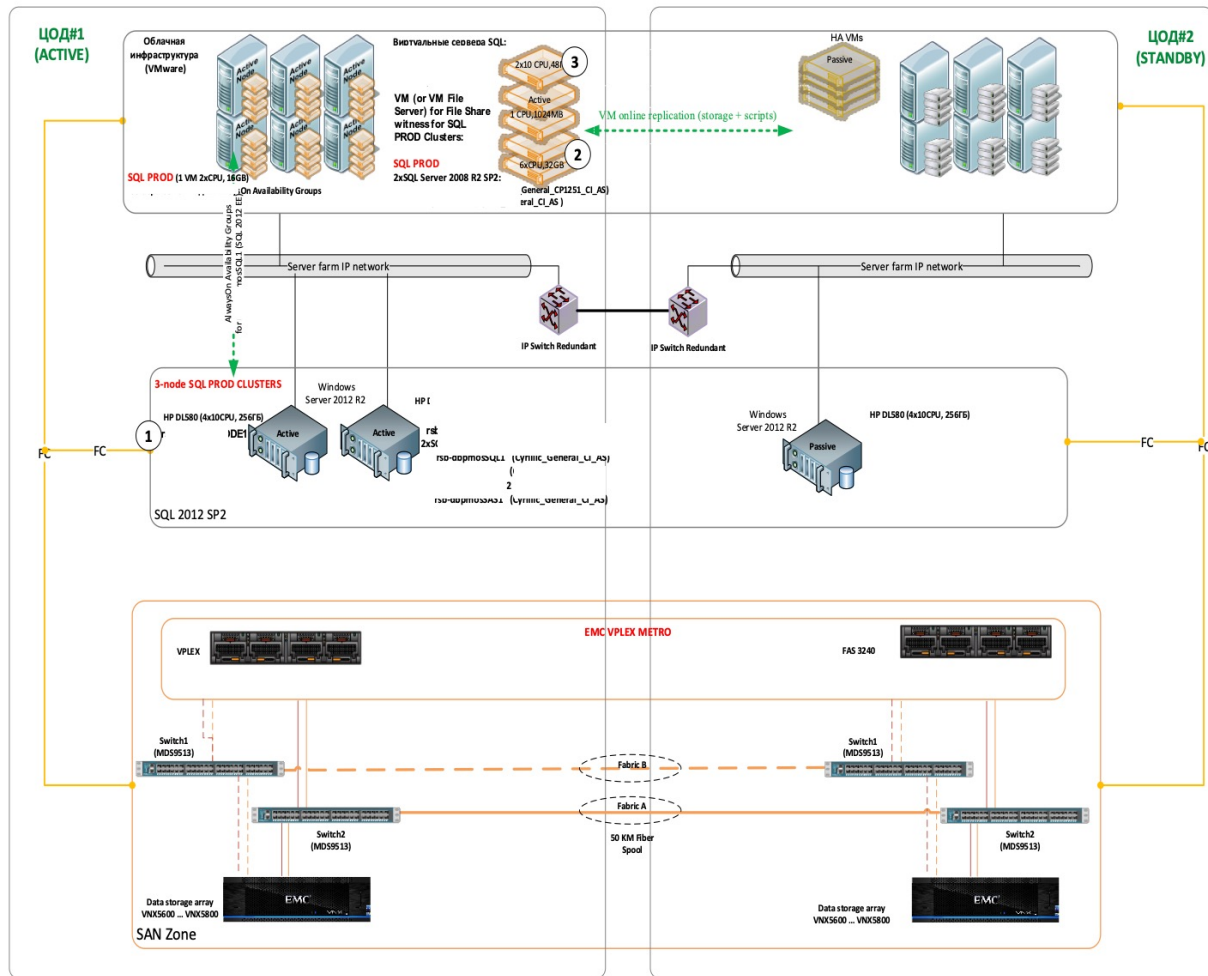


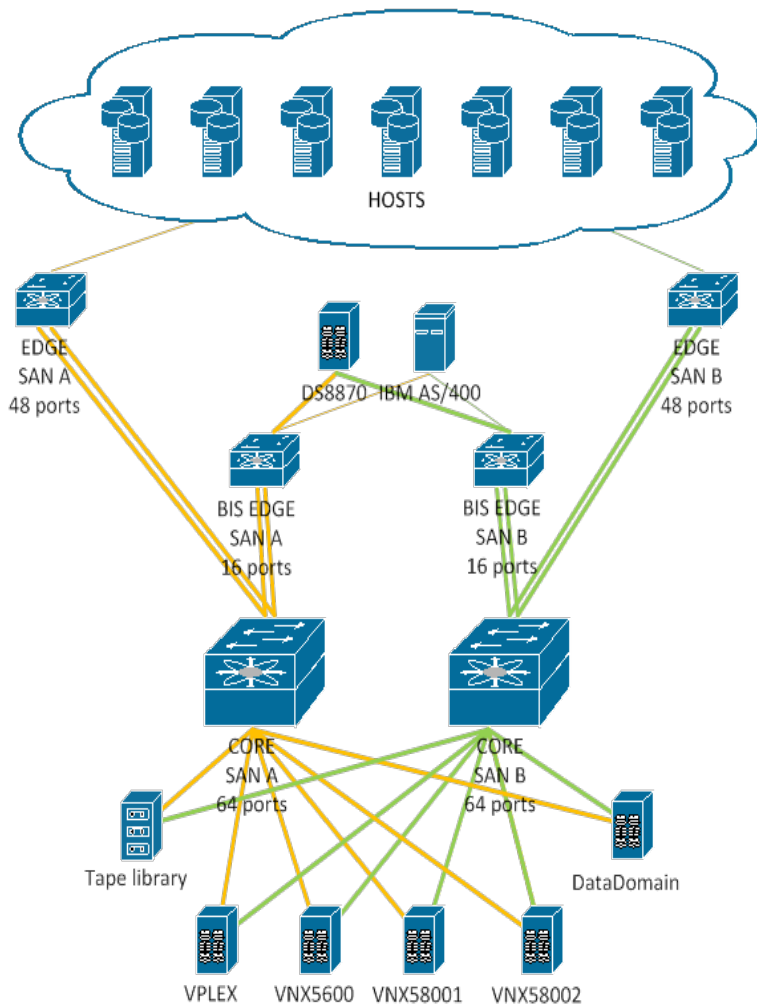






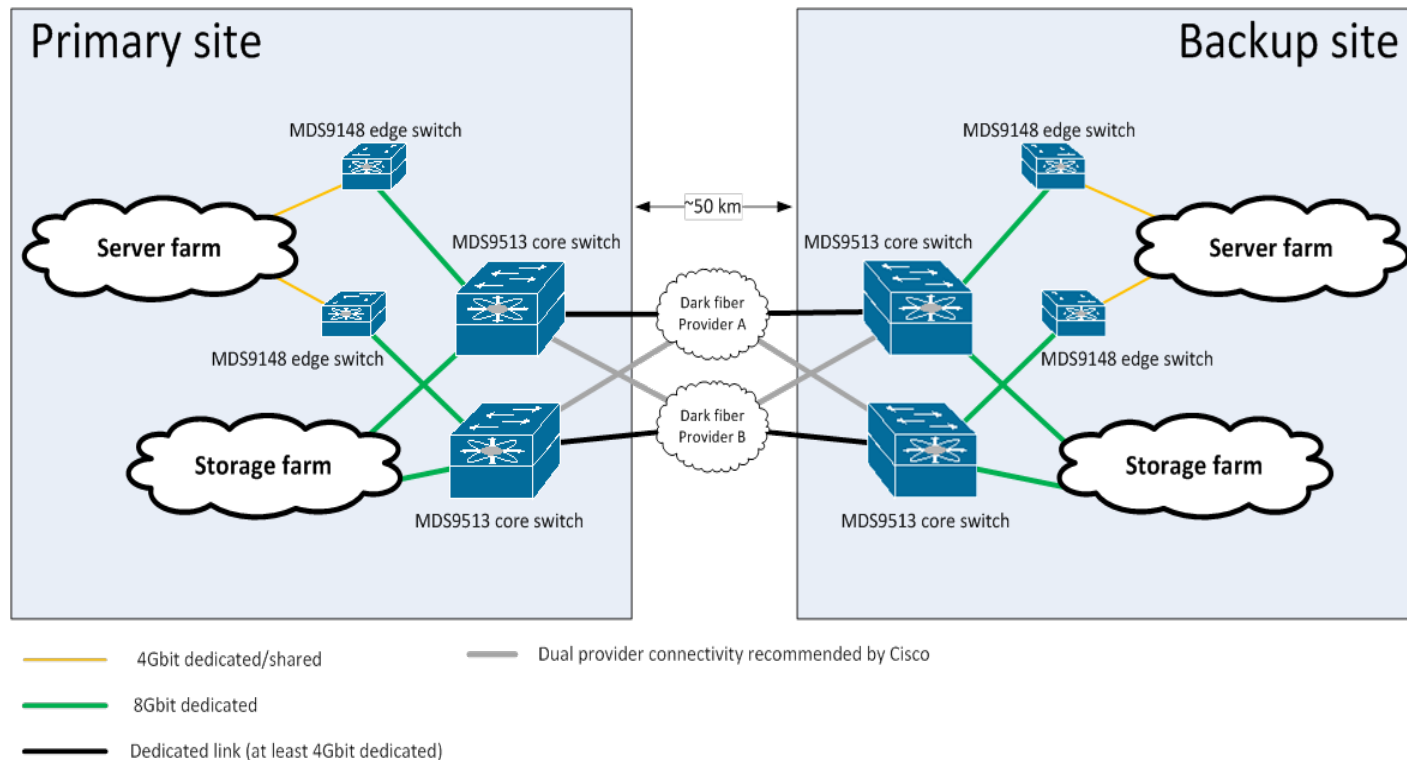
**Структурная схема катастрофоустойчивого решения для площадок №1 и №2**





Взаимодействие компонентов FC  
сети хранения данных

## Общая схема связанности и расположения элементов инфраструктуры между ЦОД#1 и ЦОД#2



# Disaster Recovery Plans

# Disaster Recovery Plans

DR-план — это документированный набор процедур и действий, которые организация должна выполнить в случае крупного сбоя, угрозы или катастрофы, чтобы восстановить операции и минимизировать потери.

Цели DR-плана:

- Восстановление критически важных систем и данных в минимальные сроки.
- Снижение финансовых и репутационных потерь.
- Обеспечение непрерывности бизнеса (Business Continuity).

- DR-план должен включать:

- Этапы восстановления
- Ответственность за каждый этап
- Точки восстановления (RPO/RTO)
- Тестирование DR:
  - Плановые и неплановые тесты
  - Симуляции сбоя
  - Анализ эффективности



# Disaster Recovery Plans

Стратегии резервного копирования (Backup) и аварийного восстановления (DR) для AI-систем имеют свои особенности, так как эти системы часто включают:

- Модели машинного обучения (ML/LLM)
- Большие объемы данных (training data, inference data)
- Высокие требования к доступности и целостности
- Интеграцию с другими сервисами (API, брокеры сообщений, СУБД)

| Компонент            | Что резервировать                                    | Рекомендации                                             |
|----------------------|------------------------------------------------------|----------------------------------------------------------|
| Модели ML            | Веса, архитектура, метаданные                        | Хранить в зашифрованном виде, с версионированием         |
| Тренировочные данные | Обучающие наборы, аннотации, метаданные              | Использовать репликацию и хранение в нескольких регионах |
| Инфраструктура       | Конфигурации, логи, настройки                        | Использовать балансировку нагрузки и кэширование ответов |
| Сервисы              | API, контейнеры, брокеры, базы данных                | Хранить в отдельной системе (MLflow, DVC, W&B)           |
| Метрики и логи       | История обучения, метрики производительности, ошибки | Проводить DR-тесты с симуляцией сбоев и потери данных    |



# Disaster Recovery Plans

|   | Компонент                    | Описание                                                                                                                |
|---|------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| 1 | Определение RTO и RPO        | RTO (Recovery Time Objective) — допустимое время простоя.<br>RPO (Recovery Point Objective) — допустимая потеря данных. |
| 2 | Критические системы и данные | Перечень сервисов, баз данных, инфраструктуры,<br>которые необходимо восстановить.                                      |
| 3 | Резервные копии и репликация | Планы резервного копирования, частота, методы, хранение,<br>восстановление.                                             |
| 4 | Стратегия восстановления     | Описание того, как и в какой последовательности восстанавливать<br>системы.                                             |
| 5 | Ответственные лица           | Четкое распределение ролей и обязанностей при сценарии сбоя.                                                            |
| 6 | Связь и коммуникация         | План уведомлений, внутренних и внешних коммуникаций.                                                                    |
| 7 | Тестирование и аудит         | Периодические проверки работоспособности плана.                                                                         |
| 8 | Документация и обновления    | Обновление плана при изменении системы, бизнес-процессов или<br>угроз.                                                  |



# Disaster Recovery Plans

| Тип резервного копирования / Метод                                     | Описание                                                      | Применение                                            | Плюсы                                      | Минусы                             | Пример / Инструменты                                      |
|------------------------------------------------------------------------|---------------------------------------------------------------|-------------------------------------------------------|--------------------------------------------|------------------------------------|-----------------------------------------------------------|
| <b>Full Backup</b><br>(Полное резервное копирование)                   | Полная копия всех данных и моделей                            | Редко, но требуется для полного восстановления        | Простота восстановления                    | Большой объём, длительное время    | —                                                         |
| <b>Incremental Backup</b><br>(Приращенческое резервное копирование)    | Сохраняются только изменения с момента последнего бэкапа      | Для экономии времени и дискового пространства         | Экономия дискового пространства и времени  | Сложность восстановления           | Резервирование только новых весов модели или новых данных |
| <b>Differential Backup</b><br>(Дифференциальное резервное копирование) | Сохраняются все изменения с момента последнего полного бэкапа | Упрощает восстановление по сравнению с приращенческим | Проще, чем incremental, при восстановлении | Больше данных, чем при incremental | —                                                         |

# Disaster Recovery Plans

| Тип резервного решения                 | Описание                                                                                                     | Применение                                        | Пример / Дополнительно                                              |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------|---------------------------------------------------------------------|
| Hot Standby (Горячее резервирование)   | Резервная система активно синхронизируется с основной                                                        | Минимизация RTO                                   | Контейнеры с моделью запущены на другом регионе, но не используются |
| Warm Standby (Тёплое резервирование)   | Резервная система периодически обновляется                                                                   | Баланс между затратами и скоростью восстановления | —                                                                   |
| Cold Standby (Холодное резервирование) | Резервная система не запущена, только данные хранятся                                                        | Для редких сценариев с низкими требованиями к RTO | —                                                                   |
| Multi-Region / Multi-Cloud DR          | Распределение нагрузки и резервных копий между несколькими регионами или облаками                            | Защита от сбоев в одном регионе                   | AI-сервис в AWS EU и Azure US                                       |
| Failover & Failback                    | Failover — переключение на резервную систему;<br>Failback — возврат на основную систему после восстановления | Для высокодоступных сервисов                      | —                                                                   |



# Документация на систему

|                                                                                                                               |     |
|-------------------------------------------------------------------------------------------------------------------------------|-----|
|  SCOM Матрица контактных лиц                 | doc |
|  SCOM_Agent_Installation_Guide_v1_6          | doc |
|  SCOM_Backup-restore_Guide_v1_7              | doc |
|  SCOM_Disaster_recovery_Guide_v1_5           | doc |
|  SCOM_Installation_Guide_v3_6                | doc |
|  SCOM_Network_flow-including_ports_v2_5      | doc |
|  SCOM_PAD_v2_6                               | doc |
|  SCOM_Start_stop_Procedure_v1_3              | doc |
|  SCOM_Testing_Procedure_Stage_2 v1_1         | doc |
|  SCOM_Testing_Protocol_Stage_2 v1_1          | doc |
|  SCOM_Update_Guide v1_1                      | doc |
|  System Acceptance Act_P12-345_v4_11_05_2012 | doc |

## Процедура запуска серверов

Включение Системы должно производиться в следующей последовательности:

1. Включение сервера резервной операционной базы данных (SQL2)  
msk-dbscomopdb3 (10.10.64.101);
2. Включение серверов операционной базы данных (SQL1)  
msk-dbscomopdb2 (10.10.64.86); (включать первым)  
msk-dbscomopdb1 (10.10.64.85);
3. Включение сервера базы данных отчетности (DW-DB)  
msk-dbscomdw (10.10.64.89);
4. Включение серверов корневого сервера управления (RMS)  
msk-isscomrms2 (10.10.64.81); (включать первым)  
msk-isscomrms1 (10.10.64.80);
5. Включение управляющих серверов (MS1,MS2)  
msk-isscomms1 (10.10.64.83);  
msk-isscomms2 (10.10.64.84);  
msk-isscomms3 (10.10.64.102);
6. Включение gateway-серверов (DMZ-GW1, DMZ-GW2)  
mskz-isscomgw1 (10.10.96.9)  
mskz-isscomgw2 (10.10.104.9);
7. Включение сервера отчетности (SRS1)  
msk-wsscomrs (10.10.64.90).



## Процедура остановки серверов

Выключение Системы должно производиться в следующей последовательности:

1. Выключение gateway-серверов (DMZ-GW1, DMZ-GW2)  
mskz-isscomgw1 (10.10.96.9);  
mskz-isscomgw2 (10.10.104.9);
2. Выключение управляющих серверов (MS1,MS2,MS3)  
msk-isscomms1 (10.10.64.83);  
msk-isscomms2 (10.10.64.84);  
msk-isscomms3 (10.10.64.102);
3. Выключение сервера отчетности (SRS1)  
msk-wsscomrs (10.10.64.90);
4. Выключение корневого управляющего сервера (RMS)  
msk-isscomrms1 (10.10.64.80); (выключать первым)  
msk-isscomrms2 (10.10.64.81);
5. Выключение сервера базы данных отчетности (DW-DB)  
msk-dbscomdw (10.10.64.89).
6. Выключение сервера операционной базы данных (SQL1)  
msk-dbscomopdb1 (10.10.64.85); (выключать первым)  
msk-dbscomopdb2 (10.10.64.86);
7. Выключение сервера резервной операционной базы данных (SQL2)  
msk-dbscomopdb3 (10.10.64.101);

# Ваши вопросы



если есть вопросы



если вопросов нет



# RAG (Retrieval-Augmented Generation)

# RAG - продвинутый LLM

**RAG (Retrieval-Augmented Generation)** — это технология искусственного интеллекта, которая улучшает ответы больших языковых моделей (LLM), сначала находя релевантную информацию во внешних источниках (например, в базах данных или интернете), а затем используя её для более точной и актуальной генерации ответа

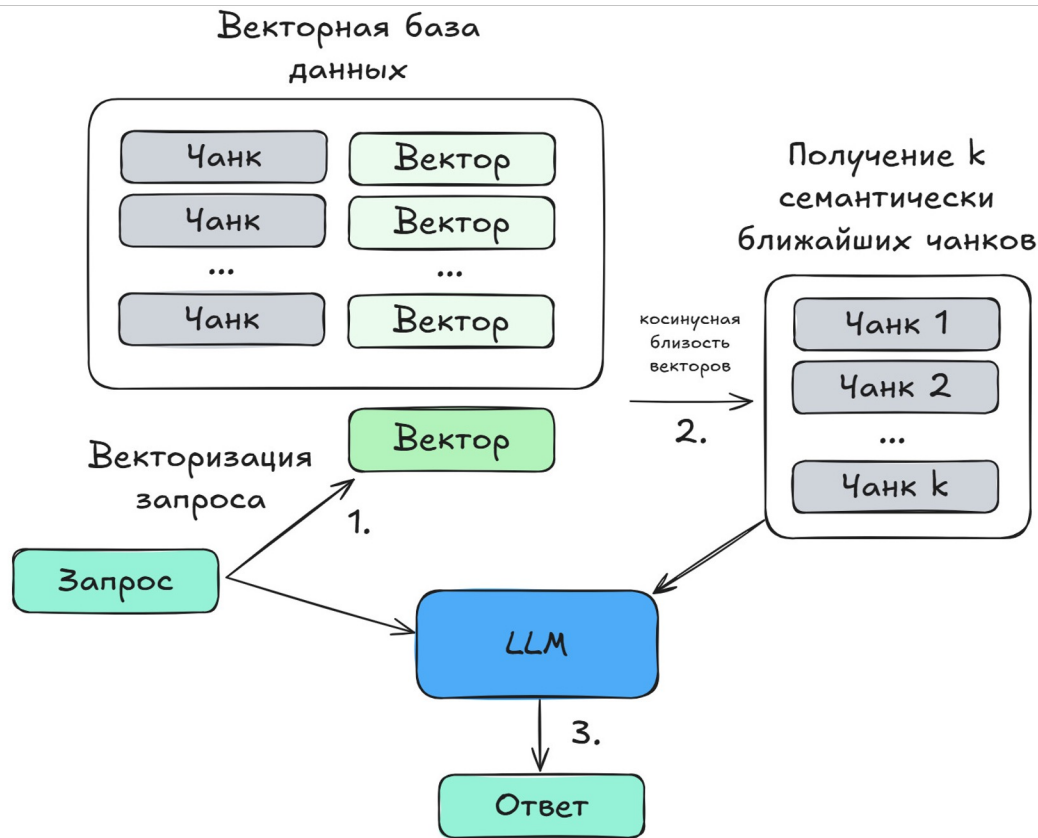
- **Поиск (Retrieval):** Когда пользователь задает вопрос, система RAG сначала ищет и извлекает наиболее релевантные фрагменты информации из внешней базы знаний. В отличие от простого поиска по ключевым словам, RAG ищет по смыслу, учитывая контекст.
- **Дополнение (Augmented):** Извлеченная информация добавляется к исходному вопросу пользователя, формируя более полный и содержательный запрос (промпт) для языковой модели.
- **Генерация (Generation):** На основе этого расширенного запроса большая языковая модель генерирует окончательный ответ, который является более точным, контекстуально верным и основанным на актуальных данных.





# Простой RAG

Запрос пользователя векторизуется (1), затем вектор сравнивается с векторами в базе данных с помощью косинусной близости(2). Топ k самых семантически близких векторов достаются из базы данных и подаются вместе с запросом пользователя в контекст LLM, которая и выдает итоговый ответ (3).



# Простой RAG

## 1. RAG-сервис (микросервис):

- Состоит из ретривера (например, FAISS, Pinecone, VectorDB) и генератора (например, LLM, тонкоструйные модели).
- Обработывает запросы, извлекает контекст и генерирует ответы.
- Может быть развернут в нескольких экземплярах для масштабирования.

## 2. Балансировщик нагрузки (Nginx)

- Распределяет трафик между экземплярами RAG-сервиса.
- Обработывает ошибки (например, 502, 503) и перенаправляет запросы к другим узлам.
- Поддерживает health checks для мониторинга доступности серверов.

## 3. База данных (VectorDB):

- Хранит векторные представления документов (например, Pinecone, Milvus, PostgreSQL с расширением pgvector).
- Нужна репликация и резервное копирование.

## 4. Кэш (Redis):

- Хранит частые запросы и ускоряет работу.
- Используется для кэширования ответов или ретриверов.

## 5. Мониторинг и логирование:

- Prometheus + Grafana для отслеживания метрик.
- ELK (Elasticsearch, Logstash, Kibana) для анализа логов.

## 6. Оркестрация (Kubernetes):

- Управляет запуском и масштабированием контейнеров.
- Обеспечивает автоматическое восстановление после сбоев.



# Архитектура AI-сервисов

# Проектирование архитектуры AI сервисов

Проектирование архитектуры AI-сервисов требует комплексного подхода, учитывающего масштабируемость, надежность, производительность, гибкость и безопасность.

## 1. Масштабируемость (Scalability)

- Цель: Повышение производительности и обработки нагрузки без снижения качества.
- Принципы:
  - о **Горизонтальное** масштабирование: Развертывание множества экземпляров сервиса (например, через **Kubernetes**, Docker Swarm) для распределения нагрузки.
  - о **Вертикальное** масштабирование: Увеличение ресурсов (CPU, RAM) у одного экземпляра, если это технически возможно.
  - о **Разделение** на компоненты: Разделение сервиса на независимые части (например, ретривер, генератор, кэш), чтобы масштабировать отдельные модули.
  - о **Асинхронная** обработка: Использование очередей (Kafka, **RabbitMQ**) для обработки задач, чтобы избежать блокировки основного потока.

RAG-сервис может быть развернут в кластере Kubernetes, где автоматически добавляются новые экземпляры при увеличении нагрузки.



# Проектирование архитектуры AI сервисов

## 2. Отказоустойчивость (Fault Tolerance)

- Цель: Обеспечить непрерывную работу сервиса даже при сбоях отдельных компонентов.
- Принципы:
  - о **Репликация**: Копирование данных и компонентов между узлами (например, репликация базы данных, кэша).
  - о **Резервные узлы**: Запуск резервных экземпляров сервиса при сбое основных.
  - о Автоматическое восстановление: Механизмы перезапуска, **перенаправления** трафика (например, **Nginx, Kubernetes**).
  - о Толерантность к ошибкам: Обработка ошибок без остановки всей системы (например, повторные запросы, логирование).

Если один экземпляр RAG-сервиса выходит из строя, Nginx автоматически перенаправляет запросы на другие узлы.



# Проектирование архитектуры AI сервисов

## 3. Производительность (Performance)

- Цель: Оптимизация времени обработки запросов и использования ресурсов.
- Принципы:
  - о Оптимизация модели: Использование **квантования**, **сжатия** моделей (например, GGUF, ONNX) для уменьшения потребления ресурсов.
  - о Кэширование: **Хранение** частых запросов (например, **Redis**, Memcached) для ускорения ответов.
  - о **Асинхронная** обработка: Разделение запросов на фоновые и синхронные задачи.
  - о **Оптимизация** инфраструктуры: Использование GPU/TPU для вычислений, **распределение** нагрузки между узлами.

Кэширование ответов RAG-сервиса в Redis позволяет ускорить обработку повторных запросов.



# Проектирование архитектуры AI сервисов

## 4. Модульность и гибкость (Modularity & Flexibility)

- Цель: Упрощение обновления, тестирования и интеграции компонентов.
- Принципы:
  - Разделение на **микросервисы**: Разделение системы на независимые модули (например, ретривер, генератор, API-гейтвей).
  - API-ориентированная архитектура: Использование **REST/gRPC** для взаимодействия между компонентами.
  - **Интеграция** с внешними сервисами: Поддержка интеграции с другими системами (например, базами данных, аналитическими инструментами).
  - Поддержка **разных** форматов данных: Обработка текста, изображений, аудио и видео.

RAG-сервис может быть разбит на отдельные микросервисы для ретривера и генератора, которые можно обновлять независимо.



# Проектирование архитектуры AI сервисов

## 5. Безопасность и защита данных (Security & Data Protection)

- Цель: Защита данных и предотвращение утечек, атак и несанкционированного доступа.
- Принципы:
  - **Шифрование** данных: Защита данных в транспорте (TLS) и в покое (AES).
  - **Контроль** доступа: Использование IAM, ролей и токенов (JWT) для ограничения доступа.
  - **Логирование** и аудит: Отслеживание всех операций для выявления аномалий.
  - **Обработка** уязвимостей: Регулярные проверки на уязвимости (например, с помощью OWASP ZAP).

Все запросы к RAG-сервису должны проходить через аутентификацию и шифрование, чтобы предотвратить несанкционированный доступ.





# Проектирование архитектуры AI сервисов

## 6. Обратная связь и обучение (Feedback & Learning)

- Цель: Улучшение модели и сервиса на основе данных и пользовательского поведения.
- Принципы:
  - о Сбор **обратной** связи: Использование метрик (например, точность, время отклика, **лайки**) для анализа эффективности модели.
  - о **Обновление** модели: Регулярное обучение модели на новых данных (например, через A/B тестирование).
  - о Логирование и **анализ**: Аналитика запросов и ошибок для выявления проблем.

Анализ ошибок в RAG-сервисе позволяет улучшить ретривер или генератор, чтобы снизить количество неправильных ответов.



# Проектирование архитектуры AI сервисов

## 7. Инфраструктура и инструменты (Infrastructure & Tools)

- Цель: Обеспечение поддержки, масштабируемости и надежности.
- Принципы:
  - о **Облачные** платформы: Использование облачных сервисов (AWS, Azure, GCP) для автоматизации и масштабирования.
  - о **Контейнеризация**: Развертывание сервисов в **Docker**/Containers для удобства управления.
  - о **Оркестрация**: Использование **Kubernetes** для управления кластерами и автоматического масштабирования.
  - о **Мониторинг** и логирование: Интеграция с Prometheus, **Grafana**, ELK для отслеживания состояния системы.

RAG-сервис может быть развернут в Kubernetes, где автоматически масштабируются ресурсы в зависимости от нагрузки.



# Ваши вопросы



если есть вопросы



если вопросов нет



# Основные паттерны отказоустойчивых систем

# Паттерны отказоустойчивых систем

**Отказоустойчивость** (fault tolerance) — это способность системы продолжать корректную работу при возникновении сбоев. В разработке отказоустойчивых систем используются различные архитектурные паттерны, которые помогают предотвратить, обнаружить и восстановиться от сбоев.

# Паттерны отказоустойчивых систем

|                                              | Описание                                                                      | Применение                                                                | Рекомендации                                                                                                                                                                                                           |
|----------------------------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Retry</b><br>(Повторная попытка)          | При сбое операции, система автоматически повторяет её выполнение              | Используется при временных сбоях (например, сбои сети).                   | <ul style="list-style-type: none"><li>- Использовать экспоненциальную задержку (exponential backoff).</li><li>- Ограничивать количество попыток.</li><li>- Учитывать idempotency (идемпотентность) операции.</li></ul> |
| <b>Circuit Breaker</b><br>(Выключатель цепи) | Мониторит частоту сбоев и временно блокирует вызовы, если сбоев слишком много | Предотвращает каскадные сбои при проблемах с зависимыми сервисами         | <ul style="list-style-type: none"><li>- Closed (работает нормально).</li><li>- Open (отключен, возвращает ошибку).</li><li>- Half-Open (проверяет, восстановилась ли система).</li></ul>                               |
| <b>Fallback</b><br>(Отказоустойчивый путь)   | При сбое операции, система использует альтернативное поведение                | Возвращение кэшированных данных, дефолтных значений или упрощённой логики | <ul style="list-style-type: none"><li>- Если сервис недоступен, возвращается статичный ответ.</li><li>- Вместо внешнего API — локальная копия данных.</li></ul>                                                        |



# Паттерны отказоустойчивых систем

|                                         | Описание                                                                                                          | Применение                                                                                                                                                    | Рекомендации                                                                                                                                                                                     |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Bulkhead</b><br>(Палубные переборки) | Ограничивает количество ресурсов, выделяемых для выполнения операции, чтобы сбой одной части не повлиял на другие | Изолирует компоненты системы от перегрузки.                                                                                                                   | <ul style="list-style-type: none"><li>- Ограниченное количество потоков или соединений для внешнего API.</li><li>- Разделение пулов ресурсов между критичными и некритичными задачами.</li></ul> |
| <b>Idempotency</b><br>(Идемпотентность) | Операция может быть выполнена один или несколько раз, результат будет одинаковым.                                 | Упрощает повторные попытки и обработку дублирующихся запросов.                                                                                                | <ul style="list-style-type: none"><li>- HTTP-методы «GET», «PUT», «DELETE» должны быть идемпотентными.</li><li>- Использование идемпотентных ключей для транзакций.</li></ul>                    |
| <b>Monitoring &amp; Health Checks</b>   | Система отслеживает своё состояние и состояние зависимостей.                                                      | <ul style="list-style-type: none"><li>- Автоматическое выявление и изоляция неисправных компонентов.</li><li>- Сбор метрик и логов для диагностики.</li></ul> |                                                                                                                                                                                                  |



# Паттерны отказоустойчивых систем

|                                                  | Описание                                                             | Применение                                                                                                                                                                        | Рекомендации                                                                                                               |
|--------------------------------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Rate Limiting</b><br>(Ограничение скорости)   | Ограничивает количество запросов, обрабатываемых за единицу времени. | Предотвращает перегрузку системы и DDoS-атаки.                                                                                                                                    | <ul style="list-style-type: none"><li>- Токен-бакет (token bucket).</li><li>- Счётчик запросов (sliding window).</li></ul> |
| <b>Load Balancing</b><br>(Балансировка нагрузки) | Распределяет трафик между несколькими экземплярами сервиса.          | Увеличивает отказоустойчивость и производительность.                                                                                                                              | <ul style="list-style-type: none"><li>- Round Robin.</li><li>- Least Connections.</li><li>- IP Hash.</li></ul>             |
| <b>Redundancy</b><br>(Резервирование)            | Создание дублирующих компонентов (сервисов, хостов, баз данных).     | <ul style="list-style-type: none"><li>- Горячее резервирование (hot standby).</li><li>- Активно-резервное (active-passive).</li><li>- Активно-активное (active-active).</li></ul> |                                                                                                                            |





# Паттерны отказоустойчивых систем

|                                                       | Описание                                                                                   | Применение                                                                                                                                             | Рекомендации                                                                                                         |
|-------------------------------------------------------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>Consensus Algorithms</b><br>(Алгоритмы консенсуса) | Позволяют группе узлов прийти к согласию при наличии сбоев.                                | <ul style="list-style-type: none"><li>- Хранение состояния в распределённых системах (например, базы данных).</li><li>- Управление кворумом.</li></ul> | <ul style="list-style-type: none"><li>- Paxos.</li><li>- Raft.</li><li>- ZAB (ZooKeeper Atomic Broadcast).</li></ul> |
| <b>Graceful Degradation</b><br>(Плавное ухудшение)    | При сбое система переключается на упрощённый режим работы.                                 | <ul style="list-style-type: none"><li>- Отключение несущественных функций.</li><li>- Вывод упрощённого интерфейса.</li></ul>                           | <ul style="list-style-type: none"><li>- Веб-сайт без JavaScript работает в базовом режиме.</li></ul>                 |
| <b>Caching</b><br>(Кэширование)                       | Хранение часто запрашиваемых данных для уменьшения нагрузки и повышения отказоустойчивости | <ul style="list-style-type: none"><li>- Кэширование ответов от внешних сервисов.</li><li>- Кэширование результатов запросов к БД.</li></ul>            |                                                                                                                      |

# Ваши вопросы

 если есть вопросы

 если вопросов нет



# Расчет RTO & RPO

# Расчет RTO & RPO

Расчёт **RTO (Recovery Time Objective)** и **RPO (Recovery Point Objective)**

**RTO** - Время, в течение которого система должна быть восстановлена после сбоя - **Не более 1 часа**

**RPO** - Максимально допустимый период потери данных — на какой момент времени должно быть восстановлено состояние системы - **Не более 4 часов**

- RTO и RPO — это ключевые метрики, которые определяют уровень отказоустойчивости и готовности к катастрофам.
- Они зависят от бизнес-требований и технологических возможностей.
- Их нужно регулярно пересматривать, особенно при изменении архитектуры или нагрузки.



# Примеры RTO & RPO

| Показатель | Вопрос                                                                    | Зависит от                                                                                                                                              | Пример                                                                                                                                                                                                            |
|------------|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RPO        | Сколько данных ты можешь потерять, не нанося значительного вреда бизнесу? | <ul style="list-style-type: none"><li>- Частота резервного копирования</li><li>- Наличие логов/транзакций</li><li>- Идемпотентность операций</li></ul>  | <ul style="list-style-type: none"><li>- Резервное копирование каждые 2 часа</li><li>- Допустимые потери: 2 часа<ul style="list-style-type: none"><li>- RPO = 2 часа</li></ul></li></ul>                           |
| RTO        | Как быстро система должна быть восстановлена после сбоя?                  | <ul style="list-style-type: none"><li>- Критичность сервиса</li><li>- Автоматизация восстановления</li><li>- Наличие резервной инфраструктуры</li></ul> | <ul style="list-style-type: none"><li>- Пользователи ждут ответ от AI-сервиса в реальном времени</li><li>- Допустимый простой: 15 минут<ul style="list-style-type: none"><li>- RTO = 15 минут</li></ul></li></ul> |

# Примеры RTO & RPO

| Уровень критичности | Примеры сфер                                                  | RTO (Время восстановления)   | RPO (Время точки восстановления) |
|---------------------|---------------------------------------------------------------|------------------------------|----------------------------------|
| Высокая             | Финансы, здравоохранение, критичные производственные процессы | От 0 до 15 минут             | От 0 до 30 минут                 |
| Средняя             | Услуги, веб-приложения                                        | От 1 часа до 4 часов         | От 1 часа до 1 дня               |
| Низкая              | Демо-приложения, аналитика                                    | От нескольких часов до суток | От 1 дня до недели               |



# Примеры RTO & RPO

| Сценарий                | RTO        | RPO        | Примечание          |
|-------------------------|------------|------------|---------------------|
| Банковская транзакция   | 0-1 минута | 0-1 минута | Высокая критичность |
| Веб-приложение          | 1 час      | 1 час      | Средняя критичность |
| AI-сервис для аналитики | 4 часа     | 1 день     | Низкая критичность  |
| Логирование и метрики   | 24 часа    | 24 часа    | Допустимы потери    |

# Расчет стоимости простоя сервиса

Сценарий:

- AI-сервис для обработки изображений в реальном времени.
- Потери: 1000 \$ в час.
- Резервное копирование моделей и данных каждые 4 часа.
- Резервная система в другом регионе (горячая).
- Восстановление занимает 1 час.

| Показатель        | Значение                                |
|-------------------|-----------------------------------------|
| RPO               | 4 часа (время между резервными копиями) |
| RTO               | 1 час (время восстановления)            |
| Стоимость простоя | $1000 \$ * 5 = 5000 \$$ (RPO + RTO)     |





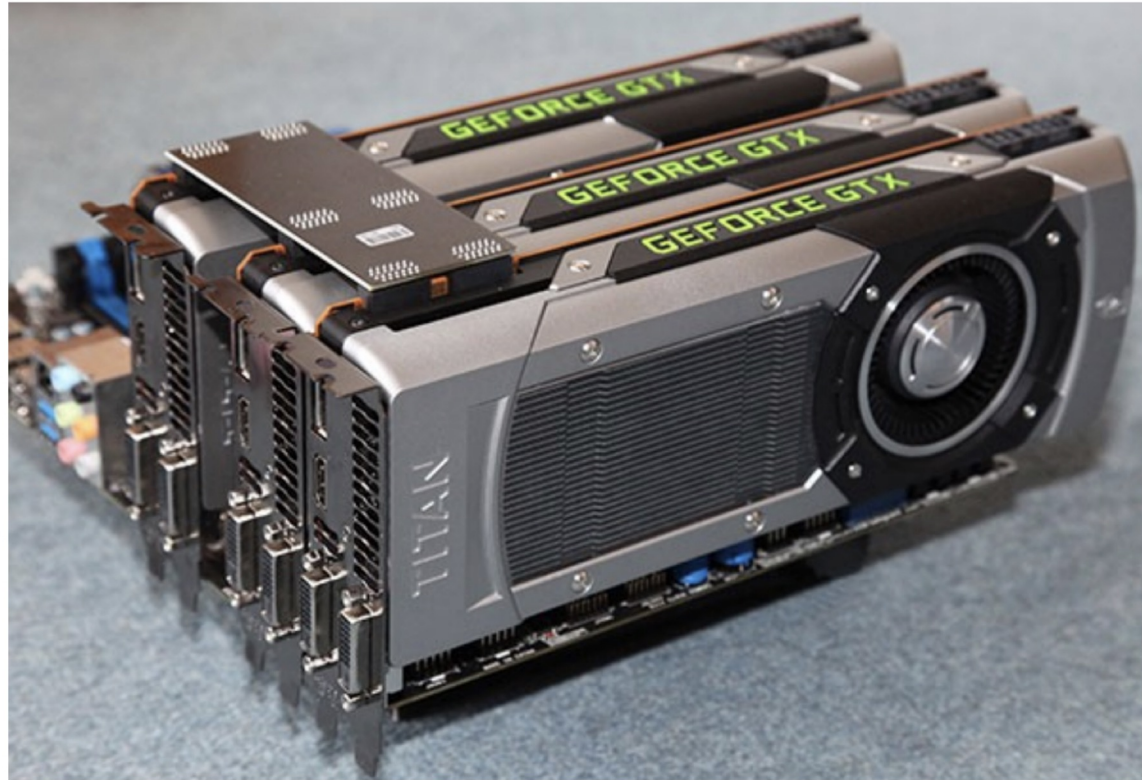
# Примеры RTO & RPO

| Сервис<br>/Компонент | Критичность | RTO    | RPO     | Стоимость<br>простоя | Комментарии                   |
|----------------------|-------------|--------|---------|----------------------|-------------------------------|
| AI-модель            | Высокая     | 15 мин | 30 мин  | 2500 \$              | Горячая резервная копия       |
| База данных          | Высокая     | 1 час  | 1 час   | 2000 \$              | Репликация в реальном времени |
| Веб-сервис           | Средняя     | 4 часа | 6 часов | 1000 \$              | Cold standby                  |



# Инфраструктура современных LLM

# GPU/TPU



# GPU TESLA V100



# Ваши вопросы

 если есть вопросы

 если вопросов нет



# Результат вебинара

Проектирование отказоустойчивых систем

Паттерны отказоустойчивости

Расчёт RTO/RPO

Примеры DRP планов

# Рефлексия